Project 3 — DocuMind-Smart-Document-Agent-Ask-Your-Documents-Anything-Safely

Title: DocuMind Smart Document Agent Ask Your Documents Anything Safely

Type: Multi-agent (3 agents) · **Stack:** Python, LangGraph, RAG (Chroma), LangChain · **Level:** Intermediate

Tagline: A 3-agent LangGraph system — Retriever, Reasoner, and Approval — that grounds every answer in your documents and pauses for a human before anything risky goes out.

> This is the **reading guide** — the story, the folder structure and why each part exists, setup, the 5 build steps, and which document to read for each idea. The other file, README.md, is the **repo README** — you copy it into your repo folder at the end and push it to GitHub. You write every folder, file and line of code yourself; the only thing already created for you is the empty repo folder `DocuMind-Smart-Document-Agent-Ask-Your-Documents-Anything-Safely/` .

Charter (what this project is)

A LangGraph app with three agents in one graph: a **Retriever** (searches your documents and judges whether what it found is good enough), a **Reasoner** (answers only from what was found, citing each file), and an **Approval** agent (approves safe drafts by itself, and pauses for a human with `interrupt()` only when a check fails). This is where "an agent" becomes "a graph" — state you can see, save, pause and resume.

Jump to: Read first · Project structure · Setup · Step 1 · Step 2 · Step 3 · Step 4 · Step 5 · Checklist · Publish

The Story — what you're actually building

Imagine a support agent asked about *your own* documents — a refund policy, a warranty, a price list — not something the model learned in training. It must answer only from what's actually in those documents. And when it's asked to *do* something ("send this to the customer"), a person should get a say first — but not on every single question, or nobody would use it.

That's what you're building. The **Retriever** goes and looks: it searches your documents and honestly judges whether the best match is close enough to answer from, instead of handing back "the closest thing" no matter how far off. The **Reasoner** writes the answer from those excerpts only, and names

the file behind each fact. The **Approval** agent is the last check: it runs a small checklist, approves safe drafts by itself, and pauses for a person only when a check fails.

Underneath, it's all one **graph**, not a loop hidden in a function. A graph can be drawn, its state saved to disk, paused mid-run, and resumed later — even after a restart.

The steps build it in order: Step 1 proves the graph mechanics. Step 2 adds saved state and a human pause. Step 3 adds the Retriever and its relevance bar. Step 4 adds the Reasoner. Step 5 swaps the always-ask pause for the Approval agent.

What you're building, in one line: a 3-agent LangGraph pipeline — search and judge, answer from the found text, approve or pause for a human.

Why this needs to exist: a model answering from training data can sound sure about your documents and be completely wrong, and letting an agent act with no human check is risky.

When you'd reach for this at a real job: answers must come from your own private documents, and some requests lead to actions that need a person's sign-off — but not every run.

How it works, mechanically: a `StateGraph` carries one typed state through the nodes. Conditional edges pick the path (search or not; grounded or not). `interrupt()` pauses the graph, a SQLite checkpointer saves the state, and `Command(resume=...)` continues it.

Why not paste all the documents into the prompt? Real document sets are far bigger than a context window, and most of the text has nothing to do with the question. **Why not ask a human every time, or never?** Always-asking doesn't scale; never-asking lets a wrong, risky action through.

Where This Fits in the 5-Project Arc

```
Project 1               → 2 agents, Python router, raw SDK
Project 2               → 2 agents, hand-built agent loop, tools
Project 3 (you are here) → 3 agents, LangGraph, RAG + human approval
                           (Retriever + Reasoner + Approval)
Project 4               → 4-5 agents, one Supervisor directs the rest
Project 5               → Project 4, made production-ready
```

Project 3's new ideas: a **graph** (saved, pausable state), **retrieval** from your own documents, and a **human in the loop** only when needed. There's still no supervisor — the routing is fixed conditional edges. Dynamic routing is Project 4's job.

What to Read First (docs this project uses)

You need to understand…	Read this	Used in
`load_config()`, `get_logger()`	Doc01 Build Task	Step 1
`prompt | model | parser`	Doc05 — LCEL basics	Steps 1, 4
`StateGraph`, state, nodes, edges	Doc09 — first graph	Step 1
`Annotated[list, operator.add]`	Doc09 Build Task	Step 1
Checkpointers, `interrupt()`, `Command(resume=...)`	Doc09 — interrupt and resume	Step 2
`SqliteSaver`, resuming in a new process	Doc09 — interrupt and resume	Step 2
Chunking, embeddings, Chroma, `retrieve()`	Doc08 Build Task	Step 3
Distance scores (lower = closer)	Doc08 — similarity	Step 3
Conditional edges	Doc09 — conditional routing	Step 3
Search only when needed	Doc10 — conditional search	Step 3
"Search found nothing"	Doc10 — empty search	Step 3
An approval pause	Doc10 — approval pause	Steps 2, 5
The whole app, as a practice run	Doc10 Build Task	All steps

Reused code, unchanged except for its folder and import lines: `config.py`, `exceptions.py`, `utils/logger.py` (Doc01), `utils/prompts.py` (Project 1), and `rag/chunking.py`, `rag/ingest.py`, `rag/retriever.py` (Doc08 — the OpenAI client moved inside each function). The graph patterns — `build_graph(checkpointer)`, `path` with `operator.add`, `interrupt()` + `Command(resume=...)`, `SqliteSaver` — are Doc09's, and the node names follow Doc10.

Project Structure (the professional layout)

Same rule as before: only folders with a real job. Project 3 adds `rag/` (the document search), `data/docs/` (the documents), and puts the graph's own files (`state.py`, `nodes.py`, `graph.py`) at the top of the package.

Names: the repo folder vs. the Python package

- **Repo folder:** `DocuMind-Smart-Document-Agent-Ask-Your-Documents-Anything-Safely/` — already created, and the exact name of the GitHub repo.

- **Python package:** `documind` (inside `src/`), imported as `from documind.graph import build_graph`.
- **Files:** `snake_case.py`, one job each; test files start with `test_`; agent files end in `_agent.py`.
- **Documents:** plain `.md` files in `data/docs/`, named for their topic (`refund_policy.md`) — the Reasoner cites these names.

The final layout (after Step 5)

```
DocuMind-Smart-Document-Agent-Ask-Your-Documents-Anything-Safely/
├── README.md               the repo's front page (copied in at the end)
├── LICENSE                 lets others reuse your code (MIT)
├── .gitignore              keeps .env, .venv/, checkpoints.sqlite out
├── .env.example            the setting names, no real values
├── .env                    your real key — never committed
├── pyproject.toml          makes src/documind an installable package
├── requirements.txt        the libraries, for pip install -r
├── src/
│   └── documind/
│       ├── __init__.py
│       ├── main.py             the terminal loop — the file you run
│       ├── config.py           Config, load_config()        (Doc01)
│       ├── exceptions.py       MissingConfigError           (Doc01)
│       ├── state.py            DocState, start_state()      (Doc09)
│       ├── nodes.py            clean_task, answer_node
│       ├── graph.py            build_graph(checkpointer)    (Doc09)
│       ├── models/
│       │   ├── __init__.py
│       │   └── llm.py          get_model() -> ChatOpenAI
│       ├── rag/
│       │   ├── __init__.py
│       │   ├── chunking.py     chunk_by_chars/paragraph     (Doc08)
│       │   ├── ingest.py       build_vector_store()         (Doc08)
│       │   ├── retriever.py    retrieve() -> list[dict]     (Doc08)
│       │   └── store.py        get_collection() — built once
│       ├── agents/
│       │   ├── __init__.py
│       │   ├── retriever_agent.py route, search, judge relevance
│       │   ├── reasoner_agent.py  grounded, cited answer
│       │   └── approval_agent.py  checklist + interrupt()
│       ├── prompts/
│       │   ├── general.txt     for non-document questions
│       │   └── reasoner.txt    "only the excerpts, cite the file"
│       └── utils/
│           ├── __init__.py
│           ├── logger.py       get_logger()                 (Doc01)
```

```
│           └── prompts.py      load_prompt(name)        (Project 1)
├── tests/
│   ├── unit/
│   │   ├── test_chunking.py        no API key needed
│   │   ├── test_routing.py         no API key needed
│   │   └── test_approval_checks.py no API key needed
│   └── integration/
│       └── test_live_graph.py      real API calls
├── scripts/
│   ├── pause_demo.py       part 1: start a run, stop at the pause
│   └── resume_demo.py      part 2: a new process finishes it
└── data/
    └── docs/               the documents DocuMind answers from
        ├── refund_policy.md
        ├── shipping.md
        ├── warranty.md
        ├── account_security.md
        └── pricing.md
```

Why each folder exists

- **`src/documind/`** — the package. The three graph files sit at its top level: `state.py` (what the graph carries), `nodes.py` (small non-agent nodes), `graph.py` (how they connect). **Why:** these three are what you read first to understand the app — Doc09's own file names.
- **`models/`** — builds the chat model. **Why:** every node gets the same model and settings; changing the model is one line.
- **`rag/`** — everything about searching documents: splitting, embedding and storing, searching. **Why:** retrieval is its own subject (Doc08) and works without the graph; keeping it apart means you can test and tune search alone.
- **`agents/`** — one file per agent: Retriever, Reasoner, Approval. **Why:** three agents, three jobs, three files — you can point at each one.
- **`prompts/`** — prompt text as `.txt` files. **Why:** the Reasoner's rules are the most-edited text in the project; they shouldn't be buried in code.
- **`utils/`** — the logger and the prompt loader, used everywhere.
- **`tests/unit/`** — routing, relevance, chunking and the approval checklist, with hand-made inputs. **Why:** fast, free, no key — run after every change.
- **`tests/integration/`** — the whole graph against the real API, every path. **Why:** costs money, so it's run on purpose.
- **`scripts/`** — the two-part pause/resume demo. **Why:** proving a resume from disk needs two separate processes — that's two scripts, not a test function.
- **`data/docs/`** — your documents. **Why:** they're *data*, not code — replace them with your own and nothing else changes.

Why each root file exists

- `README.md` — what someone sees first on GitHub.
- `LICENSE` — lets others legally reuse your code (GitHub can add MIT).
- `.gitignore` — also ignores `checkpoints.sqlite`, the saved graph states.
- `.env.example` — which settings to create, with no real values.
- `pyproject.toml` — the package's name, libraries and the `documind` command.
- `requirements.txt` — the library list for `pip install -r requirements.txt`.

Why each file exists (what and why)

Every file you'll write, in one place — the same What/Why list the documents give after each tree. The steps below build these files one at a time; come back here whenever you forget why a file exists.

- `__init__.py` (every code folder)
  - **What:** An empty file. You create one in every folder that holds Python code — nothing goes inside it.
  - **Why:** Python only treats a folder as a package when this file is there. Without it, an import like `from <package>.<folder> import ...` fails with `ModuleNotFoundError`.
- `main.py`
  - **What:** `main()` loads the config, opens `SqliteSaver.from_conn_string(DB_PATH)`, builds the graph, then loops: read a question, make a unique thread id, `graph.invoke(start_state(task), config)`, and if the result has `"__interrupt__"`, call `ask_human()` and resume with `Command(resume=decision)`. It prints the path and the answer.
  - **Why:** One starting point that owns the pause/resume conversation with the user. The graph and agents live in other files.
- `config.py`
  - **What:** A `Config` dataclass with two fields (`openai_api_key`, `log_level`), a helper `require_env(key)` that raises `MissingConfigError` when a setting is empty, and `load_config()`, which calls `load_dotenv()`, reads both settings and returns a `Config`.
  - **Why:** Every file gets its settings from one place. If the key is missing, the app stops at startup with one clear line — without this file, the missing key would show up later as a confusing API error.
- `exceptions.py`
  - **What:** A few small classes that inherit from `Exception` — at least `MissingConfigError`. Each one is just `class Name(Exception): pass`, with a docstring saying when it's raised.
  - **Why:** Code can catch one specific problem (`except MissingConfigError:`). Without named errors you'd catch every `Exception`, and real bugs would be hidden along with the expected problem.

- `utils/logger.py`
  - **What:** One function, `get_logger(name)`. It gets the logger for that name and, only the first time (`if not logger.handlers:`), adds a console handler with a timestamp format and the level from `LOG_LEVEL`.
  - **Why:** Every file logs in the same format with one line of code. Without the `if not logger.handlers:` guard, every log line would print twice.
- `utils/prompts.py`
  - **What:** One constant, `PROMPTS_DIR` (the prompts folder, from the repo root), and one function, `load_prompt(name)`, which opens `PROMPTS_DIR + "/" + name + ".txt"` and returns its text.
  - **Why:** Prompts live in text files, so you can change how an agent behaves without editing Python — and no agent has to open files itself.
- `state.py`
  - **What:** A `DocState` TypedDict (`task`, `path` with `Annotated[list[str], operator.add]`, `sources`, `grounded`, `approved`, `answer`) and `start_state(task)`, which returns a dict with every field set. Each step only adds fields.
  - **Why:** All nodes agree on the field names, `path` collects a log of every node that ran, and a run never fails because a field was missing.
- `nodes.py`
  - **What:** Two node functions: `clean_task(state)` returns the stripped task and `["clean_task"]` for the path; `answer_node(state)` runs a general-knowledge chain and returns the answer, labelled as general knowledge.
  - **Why:** One tidies the question once for every later node; the other answers questions that aren't about your documents — clearly labelled as such.
- `graph.py`
  - **What:** One function, `build_graph(checkpointer=None)`. It uses a `MemorySaver` if none is given, adds the six nodes, adds `START → clean_task`, two `add_conditional_edges` (search or skip; grounded or not), plain edges to `END`, and returns `compile(checkpointer=checkpointer)`.
  - **Why:** The whole flow is in one place you can draw. The app passes a SQLite checkpointer and the tests pass a `MemorySaver`, with no code change.
- `models/llm.py`
  - **What:** A `MODEL_NAME` constant and one function, `get_model()`, which returns `ChatOpenAI(model=MODEL_NAME, temperature=0)`.
  - **Why:** Every chain and agent gets the same model and settings from here. To try another model you change one line, not every file.
- `rag/chunking.py`
  - **What:** Two functions from Doc08: `chunk_by_chars(text, chunk_size)` cuts the text every N characters; `chunk_by_paragraph(text)` splits on blank lines and drops empty pieces.

    - **Why:** Search works on small pieces, so it can find the right paragraph instead of a whole file.
- `rag/ingest.py`
    - **What:** `build_vector_store(doc_folder, chunk_fn, collection_name)`: delete and recreate the Chroma collection, chunk every `.md` / `.txt` file, embed all chunks in **one** `embeddings.create()` call, and `collection.add()` them with ids and metadata.
    - **Why:** It builds the searchable store from `data/docs/`. Deleting first means running it twice never duplicates chunks.
- `rag/retriever.py`
    - **What:** `retrieve(collection, query, k=3)`: return `[]` for an empty store; otherwise embed the query, `collection.query(...)`, and return a list of dicts with `text`, `source_file`, `chunk_index` and `score` (a distance — lower is closer).
    - **Why:** This is the search itself, returning a checked, simple shape instead of the store's raw output.
- `rag/store.py`
    - **What:** A `StoreHolder` class with a `collection` field, one `store` object, and `get_collection()`, which builds the store only when `store.collection` is still `None`.
    - **Why:** The documents are embedded once per run — not for every question, and not at import time before the API key is loaded.
- `agents/retriever_agent.py`
    - **What:** `SEARCH_KEYWORDS`, `MAX_DISTANCE`, and five functions: `route_search(state)` → "search" or "skip"; `is_relevant(sources)` compares the smallest score with `MAX_DISTANCE`; `retrieve_node(state)` searches and stores `sources` and `grounded`; `has_grounding(state)` → "reason" or "no_grounding"; `no_grounding_node(state)` returns an honest "not found" answer.
    - **Why:** A vector store always returns something. Only a written relevance bar decides whether it's good enough — otherwise the answer would be built on nothing.
- `agents/reasoner_agent.py`
    - **What:** The node `reason_node(state)`, which returns the `answer`. In Step 3 it simply quotes the best excerpt with its file name; in Step 4 it keeps its name and output, and a model chain writes the answer instead, with the new helper `format_sources(sources)` putting each excerpt's file name in front of it.
    - **Why:** Answers come only from your documents, and each fact names its file — which the Approval agent then checks.
- `agents/approval_agent.py`
    - **What:** The node `approval_node(state)`. From Step 2 it pauses with `interrupt({answer, failed_checks})` and approves only if the person answers "yes". Step 5 adds `ACTION_WORDS` and three check functions (`cites_a_source()`, `asks_for_an_action()`, `run_checks()`), and the node first runs `run_checks()` — if nothing failed, it approves by itself.

    - **Why:** Safe answers go out on their own; anything risky or doubtful waits for a person — who is told exactly which check failed.
- `prompts/*.txt`
    - **What:** Two text files: the general prompt (with `{task}`) and the Reasoner's rules — only the excerpts, cite the file in brackets, and an exact sentence when the documents don't cover it (with `{sources}` and `{task}`).
    - **Why:** The Reasoner's rules change most often, so they live in a file you can edit easily.
- `tests/unit/*.py`
    - **What:** Three test files with hand-made inputs: the chunkers on sample text, `route_search` / `is_relevant` / `has_grounding` on fake states and score lists, and `run_checks()` on fake drafts.
    - **Why:** Every rule is checked for free, in a second, with no API key.
- `tests/integration/test_live_graph.py`
    - **What:** A `run(graph, thread_id, task)` helper and test functions that build the graph with a `MemorySaver` and run each path: grounded and auto-approved, not found, general, and an action task that pauses — resumed with "yes" in one thread and "no" in another.
    - **Why:** It proves every path through the graph works with the real API, including both human decisions.
- `scripts/pause_demo.py`
    - **What:** One `main()` that opens the SQLite checkpointer, runs a task that needs approval on a fixed thread id (`demo-1`), prints the paused draft, and exits.
    - **Why:** It is part 1 of proving that a paused run survives the program closing.
- `scripts/resume_demo.py`
    - **What:** One `main()` that opens the same SQLite file, reads `graph.get_state(config)`, and — if the run is still paused — asks yes/no and resumes it with `Command(resume=...)`.
    - **Why:** It is part 2: a brand-new process finishes the run, which proves the state really came from disk.
- `data/docs/*.md`
    - **What:** Short Markdown files, a few paragraphs each, with no heading lines.
    - **Why:** These are the only things DocuMind answers from — replace them with your own documents.

How imports work in this layout

Exactly as in Projects 1 and 2: an empty `__init__.py` in every folder with Python files (not `prompts/` or `data/`), `pip install -e .` once, imports like `from documind.agents.retriever_agent import route_search`, and **every command run from the repo root** — `data/docs/`, the prompts folder and `checkpoints.sqlite` are all found from there.

Left out on purpose (and when they come)

Folder / file in a big template	Why not here	Where it appears
`api/`, `routes.py`	A terminal app, no web server	Project 5 (Doc12)
`supervisor.py`	Routing is fixed conditional edges	Project 4 (Doc11)
`vectorstore/` on disk	5 small documents re-embed in a second at startup	When the document set grows
`evaluation/`	The tests check paths; answer quality scoring is later	Doc13
`Dockerfile`, CI	Nothing to deploy yet	Doc12-13
pytest, `conftest.py`	Plain test scripts, same as Projects 1-2	Doc13
A frontend	The approval question works in the terminal	Streamlit, when a project needs one

How the structure grows, step by step

Step	New files	Changed files
Setup	root files, `src/documind/__init__.py`	—
1	`config.py`, `exceptions.py`, `utils/logger.py`, `utils/prompts.py`, `models/llm.py`, `prompts/general.txt`, `state.py`, `nodes.py`, `graph.py`, `main.py`	—
2	`agents/approval_agent.py`, `scripts/pause_demo.py`, `scripts/resume_demo.py`	`state.py`, `graph.py`, `main.py`
3	`rag/` (4 files), `data/docs/`, `agents/retriever_agent.py`, `agents/reasoner_agent.py`, `tests/unit/test_chunking.py`, `tests/unit/test_routing.py`, `tests/integration/test_live_graph.py`	`state.py`, `graph.py`
4	`prompts/reasoner.txt`	`agents/reasoner_agent.py`
5	`tests/unit/test_approval_checks.py`	`agents/approval_agent.py`, `test_live_graph.py`

Setup (do this once, before Step 1)

Read first: Doc01 — Virtual environments, Doc01 — `.env` files. The same steps as Projects 1 and 2, with a different package name and library list.

1. Go into the repo folder, start git, and make a venv. From the curriculum's root folder (`multi-agent-ai-engineering-curriculum/`):

```
cd project_3_documind_rag_agent
cd DocuMind-Smart-Document-Agent-Ask-Your-Documents-Anything-Safely
git init
python -m venv .venv
source .venv/bin/activate          # Windows: .venv\Scripts\activate
```

2. Create `.gitignore` **before** your first commit:

```
# secrets — never pushed
.env
# the virtual environment — rebuilt from requirements.txt
.venv/
# Python's compiled cache files
__pycache__/
*.pyc
# created by "pip install -e ."
*.egg-info/
# saved graph states (Step 2) — local only
checkpoints.sqlite
```

3. Create `.env` and `.env.example`, the same as Projects 1 and 2:

```
# .env — never commit this file
OPENAI_API_KEY=sk-your-real-key-here
LOG_LEVEL=INFO
```

```
# .env.example — commit this one
# Copy this file to .env and fill in the real values. Never commit .env.
OPENAI_API_KEY=
LOG_LEVEL=INFO
```

4. Create `requirements.txt` :

```
chromadb>=1.0
langchain-core>=1.0
langchain-openai>=1.0
langgraph>=1.0
```

```
langgraph-checkpoint-sqlite>=2.0
python-dotenv>=1.0
```

New since Project 2: `langgraph` (the graph), `langgraph-checkpoint-sqlite` (saved state on disk), `chromadb` (the local vector store from Doc08). `langchain-openai` brings `openai`.

5. Create `pyproject.toml`:

```
[build-system]
requires = ["setuptools>=68"]
build-backend = "setuptools.build_meta"

[project]
name = "documind"
version = "0.1.0"
description = "A 3-agent LangGraph RAG system with a human approval gate."
requires-python = ">=3.10"
dependencies = [
    "chromadb>=1.0",
    "langchain-core>=1.0",
    "langchain-openai>=1.0",
    "langgraph>=1.0",
    "langgraph-checkpoint-sqlite>=2.0",
    "python-dotenv>=1.0",
]

[project.scripts]
# after "pip install -e .", typing "documind" runs main()
documind = "documind.main:main"

[tool.setuptools.packages.find]
# the package code lives under src/, not in the repo root
where = ["src"]
```

6. Create an empty `src/documind/__init__.py`, then install:

```
pip install -r requirements.txt
pip install -e .
```

Check it worked: `python -c "import documind; print('ok')"` prints `ok`.

Your documents: Step 3 needs 5-10 short documents in `data/docs/` — your own notes, policies or wiki pages as `.md` files, a few paragraphs each, **no heading lines**. The solution's example set is a small company's refund, shipping, warranty, account-security and pricing policies. For a bigger

public set, the Hugging Face dataset `rag-datasets/rag-mini-wikipedia` is small and made for practicing RAG.

7. Create `LICENSE` — the MIT license, so others may reuse your code. Put the current year and your own name on the Copyright line:

```
MIT License

Copyright (c) 2026 Your Name

Permission is hereby granted, free of charge, to any person obtaining a copy
of this software and associated documentation files (the "Software"), to deal
in the Software without restriction, including without limitation the rights
to use, copy, modify, merge, publish, distribute, sublicense, and/or sell
copies of the Software, and to permit persons to whom the Software is
furnished to do so, subject to the following conditions:

The above copyright notice and this permission notice shall be included in all
copies or substantial portions of the Software.

THE SOFTWARE IS PROVIDED "AS IS", WITHOUT WARRANTY OF ANY KIND, EXPRESS OR
IMPLIED, INCLUDING BUT NOT LIMITED TO THE WARRANTIES OF MERCHANTABILITY,
FITNESS FOR A PARTICULAR PURPOSE AND NONINFRINGEMENT. IN NO EVENT SHALL THE
AUTHORS OR COPYRIGHT HOLDERS BE LIABLE FOR ANY CLAIM, DAMAGES OR OTHER
LIABILITY, WHETHER IN AN ACTION OF CONTRACT, TORT OR OTHERWISE, ARISING FROM,
OUT OF OR IN CONNECTION WITH THE SOFTWARE OR THE USE OR OTHER DEALINGS IN THE
SOFTWARE.
```

Why This Project Is Good for Your Portfolio

- **What problem it solves:** an agent whose answers come from your documents, whose every step is visible in a `path` log, and which pauses for a person before anything risky — then resumes, even after a restart.
- **Why it matters:** typed state, branching, saved state and human approval are exactly what "how would you make this safe to run for real?" is asking about.
- **When you'd build something like this at a job:** document Q&A, support answers from a policy base, anything that must survive a restart or get sign-off before acting.
- **How it's built:** a `StateGraph` with typed state, conditional search, a written relevance bar, a citing Reasoner, a checklist-based Approval agent, and a SQLite checkpointer.

Problems you'll likely run into, and how to fix them:

Problem	Fix
Every question counts as "grounded"	A vector store always returns its k closest chunks — judge the **best score** against a written bar (`MAX_DISTANCE`), not "was anything returned"
Off-topic questions match a heading	A heading line alone becomes its own chunk — write documents as paragraphs, no heading lines
The Reasoner adds facts from memory	Tell it: only the excerpts, cite the file, and an exact sentence for "not covered"
A resumed run searches again, or loses its sources	Keep `sources` in the state — the checkpoint saves them
Resuming works in one script but not after a restart	`MemorySaver` lives in memory; use `SqliteSaver` and test with two separate processes
A log line prints twice after resuming	A node with `interrupt()` runs again from its first line on resume — keep real actions **after** the `interrupt()` call
`KeyError` in a node	A field wasn't in the starting state — always start runs with `start_state(task)`

Built During These Documents

08_rag → Async Python gate → 09_langgraph → 10_agent_workflows (on top of Docs 01-07)

Plan Before You Code

See 15_five_projects_index: write down Problem → Requirements → Architecture → Components → Data Flow → Implementation Plan → Coding Tasks, and **draw the graph** before writing any node.

How To Build This — Step by Step

The project grows step by step, and **after every step it runs**: Step 1 answers through a 2-node graph, Step 2 pauses every answer for you, Step 3 answers from your documents, Step 4 writes cited answers, Step 5 asks you only when needed. Build it yourself first; use the hints if you're stuck, and the solution only to compare.

Every command runs from the **repo root** with the venv active.

Step 1 — A 2-Node Graph With No Branching, Just to Prove the Mechanics

Project: DocuMind-Smart-Document-Agent-Ask-Your-Documents-Anything-Safely — Step 1 of 5

- **Where you are now:** Setup is done — nothing runs yet.
- **What's new in this step:** the shared files (config, errors, logger, prompt loader, model), and the three graph files: `state.py`, `nodes.py` (`clean_task` → `answer_node`), `graph.py`, plus `main.py`.
- **Why:** every later step adds nodes and branches to this one graph. Prove state really flows from node to node before anything gets harder to read.
- **How:** `StateGraph(DocState)` → `add_node` × 2 → `add_edge(START, ...)`, `add_edge(..., END)` → `compile()` → `graph.invoke(start_state(task))`.
- **Working after this step:** `python -m documind.main` answers questions and prints `path: clean_task -> answer_node`.

Read first: Doc09 — first graph, Doc09 Build Task.

Stuck on this step? Hint 1 · Hint 2 · Show me the solution

What to do:

1. Bring in the shared files: `config.py`, `exceptions.py`, `utils/logger.py`, `utils/prompts.py` (folder `src/documind/prompts`), and `models/llm.py` with a no-argument `get_model()`.
2. Write `state.py`: a `TypedDict` with `task`, `path: Annotated[list[str], operator.add]` and `answer`, plus `start_state(task)`.
3. Write `nodes.py`: `clean_task` (strips the task) and `answer_node` (a Doc05 chain with `prompts/general.txt`). Each returns only the fields it changes, plus its own name in `path`.
4. Write `graph.py` (`build_graph()`) and `main.py` (a question loop that prints `path` and `answer`).
5. Trace the state by hand after each node, then run it and compare.

The root files — `.gitignore`, `.env`, `.env.example`, `requirements.txt`, `pyproject.toml` and `LICENSE` — come from Setup; this step adds the files below.

Your files after Step 1:

```
src/documind/
├── __init__.py
├── main.py          (new) question loop -> graph
├── config.py        (new)
├── exceptions.py    (new)
├── state.py         (new) DocState: task, path, answer
├── nodes.py         (new) clean_task, answer_node
├── graph.py         (new) build_graph(): 2 nodes, plain edges
├── models/
│   ├── __init__.py  (new)
│   └── llm.py       (new) get_model()
├── prompts/
```

```
│   └── general.txt  (new)
└── utils/
    ├── __init__.py  (new)
    ├── logger.py    (new)
    └── prompts.py   (new)
```

Why these files (what and why):

- `main.py`
  - **What:** `main()` loads the config, opens `SqliteSaver.from_conn_string(DB_PATH)`, builds the graph, then loops: read a question, make a unique thread id, `graph.invoke(start_state(task), config)`, and if the result has `"__interrupt__"`, call `ask_human()` and resume with `Command(resume=decision)`. It prints the path and the answer.
  - **Why:** One starting point that owns the pause/resume conversation with the user. The graph and agents live in other files.
- `config.py`
  - **What:** A `Config` dataclass with two fields (`openai_api_key`, `log_level`), a helper `require_env(key)` that raises `MissingConfigError` when a setting is empty, and `load_config()`, which calls `load_dotenv()`, reads both settings and returns a `Config`.
  - **Why:** Every file gets its settings from one place. If the key is missing, the app stops at startup with one clear line — without this file, the missing key would show up later as a confusing API error.
- `exceptions.py`
  - **What:** A few small classes that inherit from `Exception` — at least `MissingConfigError`. Each one is just `class Name(Exception): pass`, with a docstring saying when it's raised.
  - **Why:** Code can catch one specific problem (`except MissingConfigError:`). Without named errors you'd catch every `Exception`, and real bugs would be hidden along with the expected problem.
- `state.py`
  - **What:** A `DocState` TypedDict (`task`, `path` with `Annotated[list[str], operator.add]`, `sources`, `grounded`, `approved`, `answer`) and `start_state(task)`, which returns a dict with every field set. Each step only adds fields.
  - **Why:** All nodes agree on the field names, `path` collects a log of every node that ran, and a run never fails because a field was missing.
- `nodes.py`
  - **What:** Two node functions: `clean_task(state)` returns the stripped task and `["clean_task"]` for the path; `answer_node(state)` runs a general-knowledge chain and returns the answer, labelled as general knowledge.
  - **Why:** One tidies the question once for every later node; the other answers questions that aren't about your documents — clearly labelled as such.

- `graph.py`
  - **What:** One function, `build_graph(checkpointer=None)`. It uses a `MemorySaver` if none is given, adds the six nodes, adds `START → clean_task`, two `add_conditional_edges` (search or skip; grounded or not), plain edges to `END`, and returns `compile(checkpointer=checkpointer)`.
  - **Why:** The whole flow is in one place you can draw. The app passes a SQLite checkpointer and the tests pass a `MemorySaver`, with no code change.
- `models/llm.py`
  - **What:** A `MODEL_NAME` constant and one function, `get_model()`, which returns `ChatOpenAI(model=MODEL_NAME, temperature=0)`.
  - **Why:** Every chain and agent gets the same model and settings from here. To try another model you change one line, not every file.
- `prompts/general.txt`
  - **What:** This text file holds the general prompt. The code loads it with `load_prompt("general")`.
  - **Why:** The Reasoner's rules change most often, so they live in a file you can edit easily.
- `utils/logger.py`
  - **What:** One function, `get_logger(name)`. It gets the logger for that name and, only the first time (`if not logger.handlers:`), adds a console handler with a timestamp format and the level from `LOG_LEVEL`.
  - **Why:** Every file logs in the same format with one line of code. Without the `if not logger.handlers:` guard, every log line would print twice.
- `utils/prompts.py`
  - **What:** One constant, `PROMPTS_DIR` (the prompts folder, from the repo root), and one function, `load_prompt(name)`, which opens `PROMPTS_DIR + "/" + name + ".txt"` and returns its text.
  - **Why:** Prompts live in text files, so you can change how an agent behaves without editing Python — and no agent has to open files itself.

Step 2 — Saved State and a Human Pause Before Anything Goes Out

Project: DocuMind-Smart-Document-Agent-Ask-Your-Documents-Anything-Safely — Step 2 of 5

- **Where you are now:** after Step 1 a 2-node graph answers questions — but nothing is saved, and every answer goes straight out.
- **What's new in this step:** the Approval agent's file, `agents/approval_agent.py`, with `approval_node` — for now it asks a person about **every** answer, using `interrupt()`. Plus a SQLite checkpointer passed into `build_graph()`, a thread id per question, `ask_human()` in `main.py`, and the two-script pause/resume demo.
- **What stays the same:** `clean_task` and `answer_node` (they never change again), the model and the prompt.

- **Why:** a graph can stop mid-run, save everything, and continue later — even after a restart. That's what lets a person approve an answer before it goes out.
- **How:** `interrupt({"answer": ..., "failed_checks": [...]})` pauses and saves; the app sees `"__interrupt__"` in the result, asks yes/no, and resumes with `graph.invoke(Command(resume=decision), config)`.
- **Working after this step:** every answer waits for your yes/no; a paused run can be finished by a different process.

Read first: Doc09 — interrupt and resume, Doc10 — approval pause.

Stuck on this step? Hint 1 · Hint 2 · Show me the solution

What to do:

1. Add `approved` to `state.py` (and to `start_state`).
2. Create `agents/approval_agent.py` (with an empty `agents/__init__.py`) and write `approval_node`: a fixed `reasons` list ("every answer is checked by a person for now"), `interrupt({"answer": ..., "failed_checks": reasons})`, then `"yes"` → approved, anything else → the answer becomes "Not approved — nothing was sent.".
3. Change `build_graph(checkpointer=None)` to take a checkpointer (a `MemorySaver` if none is given), and add `approval_node` after `answer_node`.
4. In `main.py`, open `SqliteSaver.from_conn_string("checkpoints.sqlite")`, make a thread id per question, and handle the pause with `ask_human()` (it prints the answer and the reasons).
5. Write `scripts/pause_demo.py` and `scripts/resume_demo.py`, run them one after the other, and check the second one finishes the run.

Your files after Step 2:

```
src/documind/
├── main.py            (changed) SqliteSaver, thread ids, ask_human()
├── state.py           (changed) + approved
├── graph.py           (changed) build_graph(checkpointer), + approval
├── agents/
│   ├── __init__.py    (new)
│   └── approval_agent.py (new) approval_node — asks about everything
└── (Step 1's other files, unchanged)
scripts/
├── pause_demo.py      (new) part 1: stop at the pause
└── resume_demo.py     (new) part 2: a new process resumes it
```

Why these files (what and why):

- `main.py`

  - **What:** `main()` loads the config, opens `SqliteSaver.from_conn_string(DB_PATH)`, builds the graph, then loops: read a question, make a unique thread id, `graph.invoke(start_state(task), config)`, and if the result has `"__interrupt__"`, call `ask_human()` and resume with `Command(resume=decision)`. It prints the path and the answer.
  - **Why:** One starting point that owns the pause/resume conversation with the user. The graph and agents live in other files.
- `state.py`
  - **What:** A `DocState` TypedDict (`task`, `path` with `Annotated[list[str], operator.add]`, `sources`, `grounded`, `approved`, `answer`) and `start_state(task)`, which returns a dict with every field set. Each step only adds fields.
  - **Why:** All nodes agree on the field names, `path` collects a log of every node that ran, and a run never fails because a field was missing.
- `graph.py`
  - **What:** One function, `build_graph(checkpointer=None)`. It uses a `MemorySaver` if none is given, adds the six nodes, adds `START → clean_task`, two `add_conditional_edges` (search or skip; grounded or not), plain edges to `END`, and returns `compile(checkpointer=checkpointer)`.
  - **Why:** The whole flow is in one place you can draw. The app passes a SQLite checkpointer and the tests pass a `MemorySaver`, with no code change.
- `agents/approval_agent.py`
  - **What:** The node `approval_node(state)`. From Step 2 it pauses with `interrupt({answer, failed_checks})` and approves only if the person answers "yes". Step 5 adds `ACTION_WORDS` and three check functions (`cites_a_source()`, `asks_for_an_action()`, `run_checks()`), and the node first runs `run_checks()` — if nothing failed, it approves by itself.
  - **Why:** Safe answers go out on their own; anything risky or doubtful waits for a person — who is told exactly which check failed.
- `scripts/pause_demo.py`
  - **What:** One `main()` that opens the SQLite checkpointer, runs a task that needs approval on a fixed thread id (`demo-1`), prints the paused draft, and exits.
  - **Why:** It is part 1 of proving that a paused run survives the program closing.
- `scripts/resume_demo.py`
  - **What:** One `main()` that opens the same SQLite file, reads `graph.get_state(config)`, and — if the run is still paused — asks yes/no and resumes it with `Command(resume=...)`.
  - **Why:** It is part 2: a brand-new process finishes the run, which proves the state really came from disk.

Step 3 — A Retriever Agent That Judges Its Own Search Results

Project: DocuMind-Smart-Document-Agent-Ask-Your-Documents-Anything-Safely — Step 3 of 5

- **Where you are now:** after Step 2 answers pause for approval, but they're general guesses — the graph can't read your documents.
- **What's new in this step:** Doc08's retriever (`rag/`), your documents (`data/docs/`), the Retriever agent (route, search, **judge**), an honest "not found" node, the Reasoner's file with a simple `reason_node` that quotes the best excerpt, two conditional edges, and the first tests.
- **What stays the same:** `clean_task` , `answer_node` , `approval_node` , the checkpointer and `main.py` .
- **Why:** a vector store always returns its closest chunks — even when none is relevant. Judging the best score against a written bar is what makes an answer "grounded".
- **How:** `route_search` (keywords) → `retrieve_node` (search + `is_relevant`) → `has_grounding` picks `reason_node` or `no_grounding_node` ; `reason_node` → `approval_node` .
- **Working after this step:** document questions are answered with the best excerpt and its file name; off-topic ones get "not found"; general ones skip the search.

Read first: Doc08 Build Task, Doc10 — conditional search, Doc10 — empty search, Doc09 — conditional routing.

Stuck on this step? Hint 1 · Hint 2 · Show me the solution

What to do:

1. Write 5 short documents in `data/docs/` (paragraphs, no heading lines).
2. Bring in Doc08's `chunking.py` , `ingest.py` and `retriever.py` as `rag/` , creating the OpenAI client inside each function. Add `rag/store.py` so the store is built once, on first use.
3. Add `sources` and `grounded` to the state.
4. Write `agents/retriever_agent.py` : `route_search` , `is_relevant` (best score ≤ `MAX_DISTANCE`), `retrieve_node` , `has_grounding` , `no_grounding_node` . **Write the bar down before testing.**
5. Write `agents/reasoner_agent.py` with `reason_node` : for now, the best excerpt plus its file name as the `answer` .
6. Wire the two conditional edges in `graph.py` — this is the graph's final shape.
7. Write the unit tests and the integration test; run a question the documents answer and one they don't.

Your files after Step 3:

```
src/documind/
├── state.py            (changed) + sources, grounded
├── graph.py            (changed) 2 conditional edges — final shape
├── rag/
│   ├── __init__.py     (new)
│   ├── chunking.py     (new, Doc08)
```

```
│   ├── ingest.py       (new, Doc08)
│   ├── retriever.py    (new, Doc08)
│   └── store.py        (new) get_collection()
└── agents/
    ├── retriever_agent.py (new)
    └── reasoner_agent.py  (new) reason_node — quotes the best excerpt
data/docs/              (new) 5 documents
tests/
├── unit/
│   ├── test_chunking.py   (new)
│   └── test_routing.py    (new)
└── integration/
    └── test_live_graph.py (new)
```

Why these files (what and why):

- `state.py`
  - **What:** A `DocState` TypedDict (`task` , `path` with `Annotated[list[str], operator.add]` , `sources` , `grounded` , `approved` , `answer`) and `start_state(task)` , which returns a dict with every field set. Each step only adds fields.
  - **Why:** All nodes agree on the field names, `path` collects a log of every node that ran, and a run never fails because a field was missing.
- `graph.py`
  - **What:** One function, `build_graph(checkpointer=None)` . It uses a `MemorySaver` if none is given, adds the six nodes, adds `START → clean_task` , two `add_conditional_edges` (search or skip; grounded or not), plain edges to `END` , and returns `compile(checkpointer=checkpointer)` .
  - **Why:** The whole flow is in one place you can draw. The app passes a SQLite checkpointer and the tests pass a `MemorySaver` , with no code change.
- `rag/chunking.py`
  - **What:** Two functions from Doc08: `chunk_by_chars(text, chunk_size)` cuts the text every N characters; `chunk_by_paragraph(text)` splits on blank lines and drops empty pieces.
  - **Why:** Search works on small pieces, so it can find the right paragraph instead of a whole file.
- `rag/ingest.py`
  - **What:** `build_vector_store(doc_folder, chunk_fn, collection_name)` : delete and recreate the Chroma collection, chunk every `.md` / `.txt` file, embed all chunks in **one** `embeddings.create()` call, and `collection.add()` them with ids and metadata.
  - **Why:** It builds the searchable store from `data/docs/` . Deleting first means running it twice never duplicates chunks.
- `rag/retriever.py`

    - **What:** `retrieve(collection, query, k=3)`: return `[]` for an empty store; otherwise embed the query, `collection.query(...)`, and return a list of dicts with `text`, `source_file`, `chunk_index` and `score` (a distance — lower is closer).
    - **Why:** This is the search itself, returning a checked, simple shape instead of the store's raw output.
- `rag/store.py`
    - **What:** A `StoreHolder` class with a `collection` field, one `store` object, and `get_collection()`, which builds the store only when `store.collection` is still `None`.
    - **Why:** The documents are embedded once per run — not for every question, and not at import time before the API key is loaded.
- `agents/retriever_agent.py`
    - **What:** `SEARCH_KEYWORDS`, `MAX_DISTANCE`, and five functions: `route_search(state)` → "search" or "skip"; `is_relevant(sources)` compares the smallest score with `MAX_DISTANCE`; `retrieve_node(state)` searches and stores `sources` and `grounded`; `has_grounding(state)` → "reason" or "no_grounding"; `no_grounding_node(state)` returns an honest "not found" answer.
    - **Why:** A vector store always returns something. Only a written relevance bar decides whether it's good enough — otherwise the answer would be built on nothing.
- `agents/reasoner_agent.py`
    - **What:** The node `reason_node(state)`, which returns the `answer`. In Step 3 it simply quotes the best excerpt with its file name; in Step 4 it keeps its name and output, and a model chain writes the answer instead, with the new helper `format_sources(sources)` putting each excerpt's file name in front of it.
    - **Why:** Answers come only from your documents, and each fact names its file — which the Approval agent then checks.
- `test_chunking.py`
    - **What:** Three test files with hand-made inputs: the chunkers on sample text, `route_search` / `is_relevant` / `has_grounding` on fake states and score lists, and `run_checks()` on fake drafts.
    - **Why:** Every rule is checked for free, in a second, with no API key.
- `test_routing.py`
    - **What:** Three test files with hand-made inputs: the chunkers on sample text, `route_search` / `is_relevant` / `has_grounding` on fake states and score lists, and `run_checks()` on fake drafts.
    - **Why:** Every rule is checked for free, in a second, with no API key.
- `test_live_graph.py`
    - **What:** Three test files with hand-made inputs: the chunkers on sample text, `route_search` / `is_relevant` / `has_grounding` on fake states and score lists, and `run_checks()` on fake drafts.
    - **Why:** Every rule is checked for free, in a second, with no API key.

Step 4 — A Reasoner Agent That Only Answers From What the Retriever Found

Project: DocuMind-Smart-Document-Agent-Ask-Your-Documents-Anything-Safely — Step 4 of 5

- **Where you are now:** after Step 3 document answers are one quoted excerpt — correct, but not a real answer.
- **What's new in this step:** `reason_node` keeps its name and output, but a Doc05 chain now writes the answer from **all** the excerpts; `format_sources()` and the strict prompt `prompts/reasoner.txt` are new.
- **What stays the same:** everything else — the graph, the state, the Retriever, `approval_node`, `main.py`.
- **Why:** search quality and answer quality fail for different reasons. A separate Reasoner that may only use the excerpts, and must cite them, keeps the two checkable apart.
- **How:** `format_sources()` puts each excerpt's file name in front of it → a chain with the reasoner prompt → `answer`.
- **Working after this step:** document questions get a written answer with `[file.md]` citations, still approved by you.

Read first: Doc10 Build Task (the `reason_node`), Doc05 — LCEL basics.

Stuck on this step? Hint 1 · Hint 2 · Show me the solution

What to do:

1. Write `prompts/reasoner.txt`: only the excerpts; cite each fact's file in brackets; an exact sentence for "not covered".
2. In `agents/reasoner_agent.py`, add `format_sources()` and change the inside of `reason_node` to run the chain — keep its name and its output.
3. Check several answers **by hand** against the documents.

Your files after Step 4:

```
src/documind/
├── agents/
│   └── reasoner_agent.py (changed) + format_sources(), model answer
└── prompts/
    └── reasoner.txt      (new)
```

Why these files (what and why):

- `agents/reasoner_agent.py`
  - **What:** The node `reason_node(state)`, which returns the `answer`. In Step 3 it simply quotes the best excerpt with its file name; in Step 4 it keeps its name and output, and a

model chain writes the answer instead, with the new helper `format_sources(sources)` putting each excerpt's file name in front of it.
    - **Why:** Answers come only from your documents, and each fact names its file — which the Approval agent then checks.
- `prompts/reasoner.txt`
    - **What:** This text file holds the reasoner prompt. The code loads it with `load_prompt("reasoner")`.
    - **Why:** The Reasoner's rules change most often, so they live in a file you can edit easily.

Step 5 — An Approval Agent That Only Asks a Human When It Really Needs To (final: 3 agents)

Project: DocuMind-Smart-Document-Agent-Ask-Your-Documents-Anything-Safely — Step 5 of 5

- **Where you are now:** after Step 4 you approve **every** document answer by hand — safe, but too slow to use every day.
- **What's new in this step:** the Approval agent's checklist: `cites_a_source()`, `asks_for_an_action()` and `run_checks()`. In `approval_node`, the fixed `reasons` line becomes `run_checks(state)`, and an "everything passed → approve" block is added.
- **What stays the same:** the graph, `main.py` (it already shows the reasons), the Retriever, the Reasoner, the pause/resume scripts.
- **Why:** a human check on every run gets skipped in practice; a gate that asks only when a specific check fails is one people keep.
- **How:** `run_checks()` → a list of failed checks (grounded? cites a source? asks for an action?). Empty → auto-approve. Otherwise `interrupt()` with the answer and the failed checks.
- **Working after this step:** the full project — safe answers go straight out, action requests and weak answers wait for you.

Read first: Doc10 — approval pause, and as a look ahead, Doc11 — Patterns (the Sequential row).

Stuck on this step? Hint 1 · Hint 2 · Show me the solution

What to do:

1. In `agents/approval_agent.py`, add `ACTION_WORDS`, `cites_a_source()`, `asks_for_an_action()` and `run_checks()` — one check per function, so a log line can name the one that failed.
2. In `approval_node`, replace the fixed `reasons` line with `reasons = run_checks(state)`, and add the auto-approve block before the pause.
3. Write `tests/unit/test_approval_checks.py`, and update the integration test: a document question is now **auto-approved**; an action request pauses — approve it in one run, reject it in another.
4. Run everything, and the pause/resume scripts once more.

Your files after Step 5 (final): the full tree in Project Structure. What changed:

```
src/documind/agents/
└── approval_agent.py    (changed) + the checklist, auto-approve
tests/
├── unit/test_approval_checks.py    (new)
└── integration/test_live_graph.py  (changed) + auto-approve, pause
```

Why these files (what and why):

- `agents/approval_agent.py`
  - **What:** The node `approval_node(state)`. From Step 2 it pauses with `interrupt({answer, failed_checks})` and approves only if the person answers "yes". Step 5 adds `ACTION_WORDS` and three check functions (`cites_a_source()` , `asks_for_an_action()` , `run_checks()`), and the node first runs `run_checks()` — if nothing failed, it approves by itself.
  - **Why:** Safe answers go out on their own; anything risky or doubtful waits for a person — who is told exactly which check failed.
- `integration/test_live_graph.py`
  - **What:** A `run(graph, thread_id, task)` helper and test functions that build the graph with a `MemorySaver` and run each path: grounded and auto-approved, not found, general, and an action task that pauses — resumed with "yes" in one thread and "no" in another.
  - **Why:** It proves every path through the graph works with the real API, including both human decisions.

Final Deliverable: DocuMind-Smart-Document-Agent-Ask-Your-Documents-Anything-Safely — a 3-agent LangGraph system in a proper package layout. The Retriever judges its own search. The Reasoner answers only from what was found, and cites it. The Approval agent auto-clears safe drafts and pauses for a human on the rest — and a paused run survives a restart.

Why this really is multi-agent (said plainly): three roles, each a real node with its own job and its own pass/fail judgment. There's no supervisor — the routing is fixed conditional edges, not a dynamic router. That's Project 4.

Run, Test and Configure the Final Project

The same commands and settings as the repo's README.md — once Step 5 is done, this is how the finished project is run, tested and configured.

Configuration

Variable	Required	Default	Meaning
`OPENAI_API_KEY`	yes	—	Your OpenAI API key
`LOG_LEVEL`	no	`INFO`	`DEBUG`, `INFO`, `WARNING` or `ERROR`

Other settings live in code, each in one place:

Setting	Where	Default
Chat model	`models/llm.py` → `MODEL_NAME`	`gpt-4o-mini`
Embedding model	`rag/ingest.py` → `EMBEDDING_MODEL`	`text-embedding-3-small`
Relevance bar	`agents/retriever_agent.py` → `MAX_DISTANCE`	`1.2` (lower = stricter)
Search topics	`agents/retriever_agent.py` → `SEARCH_KEYWORDS`	policy, refund, shipping, ...
Action words	`agents/approval_agent.py` → `ACTION_WORDS`	send, cancel, delete, ...

Your own documents: put `.md` or `.txt` files in `data/docs/` — plain paragraphs, no heading lines — and update `SEARCH_KEYWORDS` to match their topics. The store is rebuilt from the folder at startup.

Usage

Run from the repo root:

```
python -m documind.main     # or simply: documind
```

To see a paused run finished by a separate process:

```
python scripts/pause_demo.py     # starts a run that needs approval, exits
python scripts/resume_demo.py    # a new process finds it and finishes it
```

Running the Tests

Run from the repo root:

```
# unit tests — no API key
python tests/unit/test_chunking.py
python tests/unit/test_routing.py
python tests/unit/test_approval_checks.py
```

```
# integration tests — real API calls, needs OPENAI_API_KEY
python tests/integration/test_live_graph.py
```

The integration test runs every path: a grounded answer that's auto-approved, a question the documents don't cover, a general question, and an action request that pauses — approved in one run, rejected in another.

Checklist Before You Call This Done

- [ ] The repo folder name matches the GitHub repo name exactly
- [ ] Code lives in `src/documind/`, installed with `pip install -e .`; runs with `python -m documind.main`
- [ ] Every folder has one clear job, and you can say why it exists
- [ ] A `StateGraph` with typed state and at least one real conditional edge
- [ ] A SQLite checkpointer — a paused run is finished by a **separate** process (`pause_demo.py` → `resume_demo.py`)
- [ ] The Retriever runs only when needed, and judges the best score against a written bar
- [ ] A question the documents don't cover takes the "not found" path
- [ ] The Reasoner answers only from the found excerpts, and cites the file
- [ ] The Approval agent approves safe answers by itself, and asks a human only when a check fails — both paths tested
- [ ] `tests/unit/` pass with no API key; `tests/integration/` pass with one
- [ ] `.env` and `checkpoints.sqlite` are not in git
- [ ] You can draw the graph from the code without running it

Publish It to GitHub

1. Copy README.md from this folder into your repo folder, and put your own name in its License line.
2. Check nothing secret is staged, then commit:

```
git status    # .env, .venv/, checkpoints.sqlite must NOT appear
git add .
git commit -m "DocuMind: 3-agent LangGraph RAG with human approval"
```

3. On GitHub, create a new repo named exactly `DocuMind-Smart-Document-Agent-Ask-Your-Documents-Anything-Safely` (no README — you have one). In the repo's **Description** box, paste the **Short description** line from the top of README.md. Then:

```
git branch -M main
git remote add origin <your-repo-url>
git push -u origin main
```

Status

Not started. Track your progress in ../PROGRESS.md.

Stuck? Ask for **Hint 1** or **Hint 2** about the exact part you're stuck on. Only ask for the full code if you say **"Show me the solution."**